1. Who originally designed and created the Python programming language?

(A) Guido van Rossum
(B) Dennis Ritchie
(C) Bjarne Stroustrup
(D) James Gosling

2. Python is fundamentally categorized as which type of language?

(A) Interpreted, high-level, dynamically typed language
(B) Low-level compiled assembly language
(C) Purely functional static query language
(D) Hardware markup language

3. Which character is standardly used for writing single-line comments in Python?

(A) # (hash symbol)
(B) // (double slash)
(C) /* (slash star)
(D) -- (double hyphen)

4. How is code block structuring and nesting represented in Python instead of curly braces { }?

(A) Mandatory Whitespace Indentation
(B) Semicolons
(C) Parentheses
(D) BEGIN ... END keywords

5. Which built-in Python function displays formatted output to the standard output console?

(A) print()
(B) echo()
(C) display()
(D) write()

6. Which built-in Python function reads user keyboard input from standard input as a string?

(A) input()
(B) read()
(C) scan()
(D) get()

7. What data type does the built-in input() function always return in Python 3?

(A) str (String)
(B) int
(C) float
(D) bool

8. Which of the following is an invalid variable identifier in Python?

(A) 2nd_value (cannot start with a digit)
(B) _sensor_val
(C) voltage_1
(D) SamplingRate

9. Which of the following is a reserved keyword in Python and cannot be used as an identifier?

(A) pass
(B) variable
(C) signal
(D) frequency

10. Python variable names are strictly:

(A) Case-sensitive (e.g., 'val' and 'Val' are distinct)
(B) Case-insensitive
(C) Limited to 8 characters
(D) Prefixed with a dollar sign

11. Which built-in function returns the data type class of an object or variable in Python?

(A) type()
(B) typeof()
(C) datatype()
(D) is_type()

12. What are the four built-in fundamental scalar numeric types in Python 3?

(A) int, float, complex, bool
(B) int, double, char, byte
(C) integer, real, imaginary
(D) number, string, logic

13. How is an imaginary / complex number literal written in Python syntax?

(A) a + bj (using 'j' or 'J')
(B) a + bi
(C) complex(i)
(D) a + bk

14. Which arithmetic operator performs integer floor division, truncating the fractional remainder?

(A) // (double slash)
(B) / (single slash)
(C) % (modulus)
(D) ** (exponent)

15. Which arithmetic operator performs exponentiation (raising to a power)?

(A) **
(B) ^
(C) pow
(D) exp

16. What is the result of evaluating 17 % 5 in Python?

(A) 2 (remainder of 17 divided by 5)
(B) 3
(C) 3.4
(D) 1

17. What is the result of evaluating 2 ** 3 in Python?

(A) 8
(B) 6
(C) 5
(D) 9

18. What is the result of evaluating 15 // 4 in Python?

(A) 3
(B) 3.75
(C) 4
(D) 3.0

19. What is the result of standard division 15 / 5 in Python 3?

(A) 3.0 (always returns a float)
(B) 3
(C) 3.00
(D) 0

20. Which relational operator checks whether two operand values are strictly equal?

(A) ==
(B) =
(C) ===
(D) equals

21. Which relational operator checks whether two values are not equal?

(A) !=
(B) <>
(C) not equal
(D) ~=

22. What are the three logical boolean operators in Python?

(A) and, or, not
(B) &&, ||, !
(C) AND, OR, NOT
(D) &, |, ~

23. Which operator performs bitwise AND between two integers in Python?

(A) &
(B) and
(C) &&
(D) ^

24. Which operator performs bitwise OR between two integers in Python?

(A) |
(B) or
(C) ||
(D) +

25. Which operator performs bitwise XOR (exclusive OR)?

(A) ^
(B) xor
(C) ~
(D) **

26. Which operator performs bitwise NOT (one's complement)?

(A) ~
(B) !
(C) not
(D) -

27. Which operator shifts the bits of an integer to the left, multiplying by powers of 2?

(A) <<
(B) >>
(C) <=
(D) >=

28. What does the expression 1 << 3 evaluate to?

(A) 8 (1 * 2^3 = 8)
(B) 3
(C) 1
(D) 24

29. Which membership operator tests whether an item exists inside a sequence (string, list, tuple)?

(A) in
(B) has
(C) exists
(D) contains

30. Which identity operator tests whether two variables point to the exact same object in memory?

(A) is
(B) ==
(C) equals
(D) same

31. What does the identity test `a is not b` determine?

(A) Whether 'a' and 'b' reference different memory addresses in RAM
(B) Whether values are unequal
(C) Whether types differ
(D) Whether both are None

32. Which built-in function returns the unique integer memory address of an object in CPython?

(A) id()
(B) addr()
(C) pointer()
(D) mem()

33. Which function converts an integer, float string, or valid string into a float data type?

(A) float()
(B) to_float()
(C) real()
(D) double()

34. Which function converts a numeric string or float into an integer by truncating decimal digits?

(A) int()
(B) to_int()
(C) integer()
(D) floor()

35. What is the output of int(7.89) in Python?

(A) 7 (truncates toward zero)
(B) 8
(C) 7.0
(D) Error

36. What is the output of str(42) in Python?

(A) '42' (converts integer to string)
(B) 42
(C) "42.0"
(D) TypeError

37. Which special singleton constant represents the absence of a value or null state in Python?

(A) None
(B) null
(C) nil
(D) undefined

38. What is the boolean evaluation of bool(None) and bool(0) in Python?

(A) False and False
(B) True and False
(C) False and True
(D) True and True

39. Which parameter in print() specifies the trailing character output at the end of the line (default '\n')?

(A) end
(B) sep
(C) trail
(D) finish

40. Which parameter in print() specifies the delimiter string between multiple printed arguments?

(A) sep
(B) end
(C) split
(D) delimiter

41. What is the operator precedence order from highest to lowest in: `x = 2 + 3 * 4 ** 2`?

(A) ** (exponent) > * (multiplication) > + (addition) -> result is 2 + 3 * 16 = 2 + 48 = 50
(B) + > * > **
(C) * > ** > +
(D) Left to right strictly

42. What is the associativity of the exponentiation operator (**) in Python?

(A) Right-to-left (e.g., 2 ** 3 ** 2 evaluates as 2 ** (3 ** 2) = 2 ** 9 = 512)
(B) Left-to-right
(C) Associativity is undefined
(D) Depends on parentheses

43. What is the result of evaluating 2 ** 3 ** 2 in Python?

(A) 512 (evaluates right-to-left: 3**2 = 9, then 2**9 = 512)
(B) 64
(C) 512.0
(D) 36

44. What occurs during Short-Circuit Evaluation of the boolean expression `A and B`?

(A) If A is False, B is never evaluated because the overall expression is guaranteed to be False
(B) Both A and B always evaluate
(C) If B is True, A is skipped
(D) An exception is thrown

45. What occurs during Short-Circuit Evaluation of `A or B`?

(A) If A is True, B is never evaluated because the expression is already guaranteed to be True
(B) Both A and B always evaluate
(C) A is evaluated twice
(D) B evaluates first

46. What does the expression `10 and 20` evaluate to in Python?

(A) 20 (evaluates first operand: 10 is truthy, so returns the second operand 20)
(B) True
(C) 10
(D) 30

47. What does the expression `0 or 42` evaluate to in Python?

(A) 42 (0 is falsy, so returns the second operand)
(B) True
(C) 0
(D) False

48. What does the expression `[] or 'Default'` evaluate to in Python?

(A) 'Default' (empty list [] is falsy)
(B) []
(C) True
(D) False

49. How does Dynamic Typing in Python differ from Static Typing in C/C++?

(A) Variable types are bound to values at runtime rather than fixed to variable names at compile time
(B) Variables cannot change types
(C) Variables require explicit type declarations like int x
(D) Python lacks data types completely

50. What is Strong Typing in Python?

(A) Python prevents operations between incompatible types without explicit conversion (e.g., '10' + 5 throws a TypeError)
(B) Variables cannot be reassigned
(C) Memory is manually managed
(D) All variables are 64-bit

51. What error is raised when executing `'Voltage: ' + 5` in Python?

(A) TypeError: can only concatenate str (not 'int') to str
(B) ValueError
(C) SyntaxError
(D) ReferenceError

52. What is the correct way to concatenate string and integer in Python?

(A) f'Voltage: {5}' or 'Voltage: ' + str(5)
(B) 'Voltage: ' + 5
(C) 'Voltage: ' & 5
(D) 'Voltage: ' . 5

53. What are Formatted String Literals (F-Strings, PEP 498) introduced in Python 3.6?

(A) Strings prefixed with 'f' or 'F' that evaluate embedded Python expressions within curly braces {expression} at runtime
(B) Strings stored on flash memory
(C) Strings formatted as floating points
(D) Strings compiled to C structs

54. How do you format a floating-point voltage variable `v = 3.14159` to 2 decimal places using an f-string?

(A) f'{v:.2f}'
(B) f'{v:2d}'
(C) f'{v%2}'
(D) f'{round.2(v)}'

55. What does the str.format() method do in `'{:.3e}'.format(0.000456)`?

(A) Formats the number in scientific exponential notation with 3 decimal places ('4.560e-04')
(B) Multiplies by 1000
(C) Converts to octal
(D) Rounds to integer

56. In Python integer representation, integers have arbitrary precision, meaning:

(A) Integers can expand to occupy as much memory as necessary without integer overflow limits
(B) Integers are capped at 32 bits (2,147,483,647)
(C) Integers overflow to negative values
(D) Integers are capped at 64 bits

57. What is integer interning in CPython for small integers?

(A) CPython pre-allocates and caches singleton integer objects in the range -5 to 256 for performance reuse
(B) Compiling integers to binary
(C) Storing integers in registers only
(D) Limiting integer size to 1 byte

58. What is the result of `a = 256; b = 256; a is b` versus `x = 257; y = 257; x is y` in interactive shell?

(A) `a is b` is True (interned); `x is y` is typically False (distinct objects)
(B) Both are True always
(C) Both are False always
(D) TypeError

59. How are binary integer literals represented in Python source code?

(A) Prefixed with 0b or 0B (e.g., 0b1010 for decimal 10)
(B) Prefixed with b'
(C) Suffixed with B
(D) Enclosed in bin()

60. How are hexadecimal integer literals represented in Python?

(A) Prefixed with 0x or 0X (e.g., 0xFF for decimal 255)
(B) Prefixed with #
(C) Prefixed with h'
(D) Suffixed with H

61. How are octal integer literals represented in Python?

(A) Prefixed with 0o or 0O (e.g., 0o77 for decimal 63)
(B) Prefixed with 0
(C) Prefixed with oct'
(D) Suffixed with O

62. What does `bin(18)` return in Python?

(A) '0b10010' (string representation of binary)
(B) 10010
(C) 0x12
(D) 18.0

63. What does `hex(255)` return in Python?

(A) '0xff'
(B) 255
(C) '#FF'
(D) 0b11111111

64. What does `oct(64)` return in Python?

(A) '0o100'
(B) 100
(C) 0x40
(D) '80'

65. What is the purpose of digit separator underscores in numeric literals (e.g., `baud_rate = 115_200`)?

(A) Improves code readability without altering the numeric value
(B) Defines variable arrays
(C) Specifies floating precision
(D) Enables binary parsing

66. What is Multiple Assignment / Variable Unpacking in `v, i, r = 12.0, 0.5, 24.0`?

(A) Simultaneously binds multiple comma-separated variables to corresponding tuple elements in a single statement
(B) Creates a 3-element list
(C) Performs matrix division
(D) Assigns 24.0 to all three variables

67. How do you swap two variables `a` and `b` in Python without using a temporary third variable?

(A) a, b = b, a
(B) swap(a, b)
(C) a = b; b = a
(D) a ^= b ^= a

68. What does Chained Assignment `v1 = v2 = v3 = 0.0` accomplish?

(A) Assigns the single float reference 0.0 to all three variables simultaneously
(B) Throws a SyntaxError
(C) Compares v1, v2, and v3
(D) Assigns 0 to v3 only

69. What does Augmented Assignment `x += 5` execute in Python?

(A) Equivalent to `x = x + 5` (in-place addition for mutable objects)
(B) Compares x with 5
(C) Declares x with initial value 5
(D) Multiplies x by 5

70. What is the walrus operator (:=) introduced in Python 3.8?

(A) Assignment Expression operator: assigns a value to a variable as part of a larger expression
(B) Floor equality operator
(C) Bitwise reverse operator
(D) Pointer dereference operator

71. In the expression `if (n := len(signal)) > 1024:`, what does the walrus operator do?

(A) Calculates len(signal), binds it to variable 'n', and evaluates whether 'n' is greater than 1024 in one step
(B) Throws a SyntaxError
(C) Checks if n equals 1024
(D) Initializes a while loop

72. What is the Difference between an Expression and a Statement in Python?

(A) An expression evaluates to a value (e.g., `3 + 4`, `x * y`); a statement represents an executable action (e.g., assignment, `if`, `while`)
(B) Statements return values; expressions do not
(C) Expressions end with semicolons
(D) There is zero difference

73. What does the pass statement represent in Python syntax?

(A) A null operation placeholder where code is syntactically required but no action should be executed
(B) Terminates the program immediately
(C) Skips to the next loop iteration
(D) Throws an exception

74. What does the del statement do in `del x`?

(A) Deletes the variable reference 'x' from the current local or global namespace
(B) Sets x to 0
(C) Sets x to None
(D) Deletes the file named x

75. What happens when you evaluate a float addition `0.1 + 0.2 == 0.3` in Python?

(A) False (due to IEEE 754 binary floating-point representation: 0.1 + 0.2 is ~0.30000000000000004)
(B) True
(C) TypeError
(D) ZeroDivisionError

76. What module should be used in Python when exact decimal arithmetic is required without binary floating-point rounding errors?

(A) decimal (from decimal import Decimal)
(B) math
(C) fractions
(D) numpy

77. How do you accurately test whether two floating-point numbers are approximately equal in Python?

(A) math.isclose(a, b, rel_tol=1e-9)
(B) a == b
(C) abs(a) == abs(b)
(D) a is b

78. What does the math.frexp(x) function return for an engineering float?

(A) A tuple (m, e) where x = m * (2 ** e), representing the mantissa and binary exponent
(B) The fractional and integer parts
(C) Square root and log
(D) Euler's constant

79. What does the math.modf(x) function return?

(A) A tuple (fractional_part, integer_part) of the float x
(B) Modulus and floor
(C) Quotient and remainder
(D) Trigonometric mod

80. What is the output of the bitwise calculation `~12` in two's complement Python representation?

(A) -13 (since ~x = -(x + 1) in two's complement representation)
(B) -12
(C) 11
(D) -11

81. What does the expression `12 & 10` evaluate to in Python (12 is 0b1100, 10 is 0b1010)?

(A) 8 (0b1000 = 8)
(B) 14
(C) 2
(D) 10

82. What does the expression `12 | 10` evaluate to in Python?

(A) 14 (0b1110 = 14)
(B) 8
(C) 2
(D) 22

83. What does the expression `12 ^ 10` evaluate to in Python?

(A) 6 (0b0110 = 6)
(B) 8
(C) 14
(D) 2

84. In digital circuit logic modeling, how do you toggle the k-th bit (0-indexed) of an integer `mask`?

(A) mask ^= (1 << k)
(B) mask &= ~(1 << k)
(C) mask |= (1 << k)
(D) mask >>= k

85. How do you set the k-th bit of an integer `mask` to 1?

(A) mask |= (1 << k)
(B) mask &= (1 << k)
(C) mask ^= (1 << k)
(D) mask = 1 << k

86. How do you clear (set to 0) the k-th bit of an integer `mask`?

(A) mask &= ~(1 << k)
(B) mask |= (1 << k)
(C) mask ^= (1 << k)
(D) mask = ~(1 << k)

87. How do you check whether the k-th bit of an integer `mask` is 1?

(A) bool(mask & (1 << k))
(B) bool(mask | (1 << k))
(C) mask == k
(D) mask[k] == 1

88. What is the effect of Brian Kernighan's bit manipulation trick `n &= (n - 1)`?

(A) Clears the lowest set bit (rightmost 1-bit) of integer 'n'
(B) Doubles n
(C) Inverts all bits of n
(D) Rounds n to nearest power of 2

89. How do you determine if a positive integer 'n' is a power of 2 using bitwise operators?

(A) (n > 0) and ((n & (n - 1)) == 0)
(B) n & 2 == 0
(C) n % 2 == 0
(D) n >> 1 == 0

90. What does `int('11010', 2)` do in Python?

(A) Parses the binary string '11010' and returns decimal integer 26
(B) Throws ValueError
(C) Returns binary string
(D) Returns 11010.0

91. What does `int('2A', 16)` evaluate to in Python?

(A) 42 (2 * 16 + 10 = 42)
(B) 20
(C) 210
(D) 40

92. What is the purpose of the sys.getsizeof(obj) function in Python?

(A) Returns the memory consumption of an object in RAM in bytes
(B) Returns character count
(C) Returns array length
(D) Returns CPU clock cycles

93. Why does an empty integer `0` consume 24 or 28 bytes in CPython rather than 4 or 8 bytes?

(A) CPython integers are heap-allocated PyObject structs containing reference count, type pointer, and size fields
(B) Python wastes memory intentionally
(C) It allocates a 256-bit buffer
(D) It contains ASCII strings

94. What is Garbage Collection in Python primarily governed by?

(A) Reference Counting combined with a cyclic garbage collector (generational GC)
(B) Pure mark-and-sweep only
(C) Manual free() pointers
(D) Operating system page swapping

95. What happens when an object's reference count drops to zero in Python?

(A) Its memory is immediately deallocated and reclaimed by CPython
(B) It moves to generation 2
(C) It waits for program exit
(D) It throws a MemoryError

96. What module in Python allows inspecting and tuning the cyclic garbage collector?

(A) gc (import gc)
(B) sys
(C) mem
(D) heap

97. What is the consequence of modifying a global variable inside a function without declaring `global var_name`?

(A) Python treats it as a local variable; reading it before assignment raises an UnboundLocalError
(B) It updates the global variable automatically
(C) It throws a SyntaxError at compile time
(D) It deletes the variable

98. What does `sys.float_info` describe in Python?

(A) A struct sequence holding low-level hardware IEEE 754 float parameters (epsilon, max, min, dig)
(B) The version of Python
(C) The size of integers
(D) GPU driver status

99. What is the value of `sys.float_info.epsilon` on standard 64-bit systems?

- **(A)** Approximately 2.220446049250313e-16 (the difference between 1.0 and the least value greater than 1.0)
- **(B)** 1e-6
- **(C)** 1e-32
- **(D)** 0.0

100. In Python 3, what does the expression `eval('3 + 5 * 2')` do, and why should it be avoided with untrusted input?

- **(A)** Parses and executes the string as arbitrary Python code; dangerous because attackers can execute arbitrary system commands via `__import__('os').system()`
- **(B)** Validates string syntax safely
- **(C)** Converts string to JSON
- **(D)** Formats strings to math LaTeX

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the architecture of the Python programming language: CPython virtual machine, bytecode compilation (.pyc), and the interpreter execution cycle.

Q2. Compare Interpreted vs Compiled languages, and analyze Python's dynamic typing vs C/C++'s static typing in embedded systems engineering.

Q3. Detail Python's scalar data types: int, float, complex, and bool, explaining their internal memory representations in CPython.

Q4. Explain Arithmetic, Relational, and Logical Operators in Python, detailing operator precedence and short-circuit evaluation.

Q5. Analyze Bitwise Operators in Python (&, |, ^, ~, <<, >>) and illustrate their application in low-level microcontroller register manipulation.

Q6. Explain Variable Assignment, Dynamic Typing, Multiple Assignment, and the Walrus Operator (:=) in Python.

Q7. Explain Formatted I/O in Python: print() parameters (sep, end, file, flush), str.format(), and Modern F-Strings.

Q8. Explain Type Conversion in Python, contrasting Implicit Type Coercion with Explicit Casting, detailing edge cases.

Q9. Explain CPython's Memory Management model: Object Reference Counting, Generational Garbage Collection, and Object Interning.

Q10. Describe IEEE 754 Floating-Point representation in Python, floating-point rounding errors, and the math.isclose() solution.

Q11. Explain Python Statements vs Expressions, Simple Statements, Compound Statements, and the LEGB variable lookup rule.

Q12. Explain Complex Numbers in Python and demonstrate their application in solving AC Electrical Circuit Impedance (R-L-C).

Q13. Describe Python's math and cmath standard libraries, highlighting key functions for electronics engineering computations.

Q14. Detail Python's sys and os modules, explaining sys.argv, sys.exit(), os.environ, and command-line automation in electronics testing.

Q15. Explain the Python Virtual Environment architecture (venv), detailing site-packages isolation and requirements.txt dependency management.

Q16. Analyze Python's Garbage Collection mechanism in detail, explaining how circular references are collected via Generational GC.

Q17. Explain Integer Interning and String Interning in CPython, analyzing their performance benefits and identity test (is vs ==) implications.

Q18. Explain Python's Decimal and Fraction modules, demonstrating how they eliminate precision loss in engineering calculations.

Q19. Detail Error and Exception Handling fundamentals in Python: try, except, else, finally, and custom exception classes.

Q20. Explain how Python script files are executed from the command line, detailing shebang lines (#!/usr/bin/env python3) and `if __name__ == '__main__':`.

Q21. Write a complete Python script to calculate Ohm's Law and Power Dissipation given user inputs, handling ValueError exceptions robustly.

Q22. Write a Python program to perform Bitwise Bit-Masking operations for an 8-bit GPIO Port Register (setting, clearing, toggling, and reading pins).

Q23. Write a Python script to compute the Equivalent Resistance of N resistors in Series and Parallel configurations.

Q24. Write a Python script to compute the frequency, period, and wavelength of an electromagnetic carrier wave given frequency in MHz.

Q25. Demonstrate how to convert an analog ADC reading (0?1023 on a 10-bit ADC) to its corresponding physical Voltage (0?5V) using Python.

Q26. Write a Python program that formats a telecommunication engineering telemetry log using modern f-strings with alignment and padding.

Q27. Write a Python script to compute the decibel (dB) gain and attenuation of voltage and power ratios.

Q28. Demonstrate how to parse command-line arguments using Python's sys.argv to configure serial baud rates and COM ports.

Q29. Explain the difference between `is` and `==` operators with mutable and immutable data structures, writing code tests.

Q30. Analyze memory usage of Python data types using sys.getsizeof() and compare integer memory footprints across Python and C.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1: [A]
Q2: [A]
Q3: [A]
Q4: [A]
Q5: [A]
Q6: [A]
Q7: [A]
Q8: [A]
Q9: [A]
Q10: [A]
Q11: [A]
Q12: [A]
Q13: [A]
Q14: [A]
Q15: [A]
Q16: [A]
Q17: [A]
Q18: [A]
Q19: [A]
Q20: [A]
Q21: [A]
Q22: [A]
Q23: [A]
Q24: [A]
Q25: [A]
Q26: [A]
Q27: [A]
Q28: [A]
Q29: [A]
Q30: [A]
Q31: [A]
Q32: [A]
Q33: [A]
Q34: [A]
Q35: [A]
Q36: [A]
Q37: [A]
Q38: [A]
Q39: [A]
Q40: [A]
Q41: [A]
Q42: [A]
Q43: [A]
Q44: [A]
Q45: [A]
Q46: [A]
Q47: [A]
Q48: [A]
Q49: [A]
Q50: [A]
Q51: [A]
Q52: [A]
Q53: [A]
Q54: [A]
Q55: [A]
Q56: [A]
Q57: [A]
Q58: [A]
Q59: [A]
Q60: [A]
Q61: [A]
Q62: [A]
Q63: [A]
Q64: [A]
Q65: [A]
Q66: [A]
Q67: [A]
Q68: [A]
Q69: [A]
Q70: [A]
Q71: [A]
Q72: [A]
Q73: [A]
Q74: [A]
Q75: [A]
Q76: [A]
Q77: [A]
Q78: [A]
Q79: [A]
Q80: [A]
Q81: [A]
Q82: [A]
Q83: [A]
Q84: [A]
Q85: [A]
Q86: [A]
Q87: [A]
Q88: [A]
Q89: [A]
Q90: [A]
Q91: [A]
Q92: [A]
Q93: [A]
Q94: [A]
Q95: [A]
Q96: [A]
Q97: [A]
Q98: [A]
Q99: [A]
Q100: [A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION D: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS [illegible]

[illegible]

I

[illegible] register manipulation.

Introduced in Python 3.8 to perform Assignment Expressions within conditionals and loops:

```python
# Read serial sensor buffer while data exists:
while (packet := serial_port.read()) != b'':
process_sensor_packet(packet)
```

Avoids duplicate reads and eliminates redundant assignments outside loop headers.

>> Solution to Question 8: [illegible]

- `int(x)`: Truncates float decimal digits (`int(9.99)` -> `9`) or parses valid numeric strings (`int('101', 2)` -> `5`). Throws `ValueError` if parsing fails.
- `float(x)`: Converts strings/integers to float (`float('3.3')` -> `3.3`).
- `str(x)`: Serializes any object into human-readable string form (`str(100)` -> `'100'`).
- `bool(x)`: Evaluates truthiness (0, None, empty containers evaluate to False; all others True).

[illegible]

I

[illegible]

```
Z_L = 1j * omega * L
Z_C = 1 / (1j * omega * C)

# Total series impedance Z_total
Z_total = Z_R + Z_L + Z_C
magnitude_Z = abs(Z_total)
phase_rad = cmath.phase(Z_total)
phase_deg = math.degrees(phase_rad)

print(f'Impedance: {Z_total:.2f} Ohms')
print(f'|Z|: {magnitude_Z:.2f} Ohms, Phase Angle: {phase_deg:.2f} deg')
```

[illegible] testing.

- `os.system(cmd)`: Executes shell commands (e.g. flashing microcontrollers via avrdude/esptool).

[illegible]

2. Cycle Detection: Periodically subtracts internal reference links between objects in Gen 0. If an isolated graph has zero external references, it is swept and freed.
3. Promotion: Objects surviving collection are promoted to older generations, which are scanned less frequently, optimizing CPU overhead.

[illegible]

2. The `fractions` Module (Exact Rational Numbers):

```python
from fractions import Fraction

# Voltage divider resistor ratio R1/(R1 + R2)
r_ratio = Fraction(100, 300) # Automatically reduces to 1/3
gain = Fraction(1, 2) + r_ratio # 1/2 + 1/3 = 5/6
print(gain) # 5/6
print(float(gain)) # 0.8333333333333334
```

[illegible] and `if name

1. The Shebang Line (`#!/usr/bin/env python3`):
Placed on the very first line of a Python script in Unix/Linux/Raspberry Pi OS. Instructs the operating system loader to locate the Python 3 binary in the user's environment and execute the script directly from the terminal without typing `python3 script.py`.

2. The `if __name__ == '__main__':` Idiom:
- When a script is executed directly from the terminal (`python my_script.py`), the CPython runtime assigns the special built-in string `'__main__'` to the module's `__name__` variable.
- When the script is imported by another module (`import my_script`), `__name__` is set to the filename `'my_script'`.
- By enclosing execution logic inside `if __name__ == '__main__':`, test code and main routines only run when executed directly, preventing accidental execution when imported by test suites.

>>Solution to Question 22: [illegible]

~~~
def __init__(self, initial_val=0x00):
self.port = initial_val & 0xFF # Enforce 8-bit mask

def set_pin(self, pin):
self.port |= (1 << pin)

def clear_pin(self, pin):
self.port &= ~(1 << pin)

def toggle_pin(self, pin):
self.port ^= (1 << pin)

def read_pin(self, pin):
return 1 if (self.port & (1 << pin)) else 0

def display(self):
print(f'PORT Value: 0x{self.port:02X} | Binary: 0b{self.port:08b}')

gpio = GPIORegister(0b00000000)
gpio.set_pin(2) # Set Pin 2 HIGH
gpio.set_pin(5) # Set Pin 5 HIGH
gpio.display() # 0b00100100 (0x24)
gpio.toggle_pin(2) # Toggle Pin 2 LOW
gpio.display() # 0b00100000 (0x20)
print('Pin 5 State:', gpio.read_pin(5)) # 1
```
~~~

>>Solution to Question 22:

>> Problem Statement: Write a Python script to compute the Equivalent Resistance of N resistors in Series and Parallel configurations.

[illegible]

```python
def generate_telemetry_table(data):
print('=' * 65)
print(f'{"NODE":<10} | {"VOLT (V)":<10} | {"TEMP (?C)":<10} | {"RSSI (dBm)":<12} |
{"STATUS":<8}')
print('=' * 65)
for node, v, temp, rssi, status in data:
print(f'{node:<10} | {v:<10.2f} | {temp:<10.1f} | {rssi:<12.1f} | {status:<8}')
print('=' * 65)

nodes_data = [
('Node_01', 3.289, 42.67, -68.4, 'ACTIVE'),
('Node_02', 3.102, 55.10, -82.1, 'WARN'),
('Node_03', 2.945, 38.45, -94.7, 'CRIT')
]
generate_telemetry_table(nodes_data)
```

>> **Solution to Question D26:** [illegible] baud rates and COM ports.

```
# Default configuration
port = 'COM3'
baud = 9600

if len(sys.argv) > 1:
port = sys.argv[1]
if len(sys.argv) > 2:
try:
baud = int(sys.argv[2])
except ValueError:
print(f'Warning: Invalid baud "{sys.argv[2]}", defaulting to 9600.')

print(f'Connecting to Instrument on Port: {port} at {baud} baud.')

if __name__ == '__main__':
parse_serial_args()
```

[illegible] and C.

```
print(f'Size of integer 0 : {sys.getsizeof(0)} bytes') # 28 bytes
print(f'Size of integer 1 : {sys.getsizeof(1)} bytes') # 28 bytes
print(f'Size of float 0.0 : {sys.getsizeof(0.0)} bytes') # 24 bytes
print(f'Size of empty string "": {sys.getsizeof("")} bytes') # 49 bytes
print(f'Size of empty list [] : {sys.getsizeof([])} bytes') # 56 bytes
print(f'Size of None object : {sys.getsizeof(None)} bytes') # 16 bytes
```

Analysis:
In C, an `int` occupies strictly 4 bytes on stack memory. In CPython, an integer occupies 28 bytes because it is a full `PyLongObject` containing reference counts and virtual machine dispatch pointers.
